

DOCUMENTO DE ARQUITETURA E IMPLANTAÇÃO DE SISTEMA

VERITAS CODE

Integrantes: Guilherme Leite, Guilherme Machado, Leonardo Matiusso, Marina Duque, Nathan Damasio, Samuel Martins

1 INFORMAÇÕES DO SISTEMA E ARQUITETURA

1.1 Visão Geral da Arquitetura

A aplicação "Radar Cidadão" utiliza uma arquitetura baseada em renderização do lado do servidor (Server-Side Rendering - SSR). As páginas front-end são geradas dinamicamente a cada requisição do usuário, utilizando as tecnologias Python, Flask e templates Jinja2. O fluxo de dados segue a ordem: Navegador do Usuário -> Servidor Web (Flask) -> Lógica da Aplicação -> Banco de Dados Relacional (MySQL/MariaDB). O gerenciamento de conteúdo é realizado de forma autônoma por um processo de Extração, Transformação e Carga (ETL), que consome dados da API pública da Câmara dos Deputados.

1.2 Pilha Tecnológica (Stack) e Dependências

O projeto é estruturado com base nas seguintes tecnologias:

- a) Front-end: HTML5, CSS3, JavaScript e Bootstrap.
- b) Back-end: Python 3 e microframework Flask.
- c) Processamento de Dados: Bibliotecas Pandas, Requests, Aiohttp e Asyncio.
- d) Integrações Externas: API da Câmara dos Deputados (dados parlamentares).

1.3 Infraestrutura de Armazenamento de Ativos e Conteúdo

- a) Hospedagem de Mídia: Ativos estáticos (CSS, logotipos) são armazenados no diretório local "/static". Imagens pesadas, como as fotografias dos parlamentares, são acessadas diretamente por meio de URLs fornecidas pela API da Câmara.
- b) Banco de Dados: Um banco de dados relacional MySQL/MariaDB armazena todos os dados estruturados processados pela rotina ETL.

2 GUIA DE IMPLANTAÇÃO PASSO A PASSO

2.1 Requisitos e Verificações Preliminares

Para a correta implantação, os seguintes requisitos devem ser atendidos:

- a) Conta de Hospedagem: Acesso ao servidor de produção.
- b) Instância de Banco de Dados: Servidor MySQL ou MariaDB ativo.
- c) Controle de Versão: Acesso ao repositório Git do projeto.
- d) Domínio: Domínio registrado e ativo.

2.2 Configuração do Ambiente e Implantação

2.2.1 Obtenção do Código-Fonte

Acesse o servidor de produção via terminal e efetue a clonagem do repositório:

```
git clone https://github.com/ninadunquehc/API.git  
cd API
```

2.2.2 Estruturação do Banco de Dados

Execute o script SQL fornecido para criar as tabelas necessárias no banco de dados MySQL:

```
mysql -h <host_do_banco> -u <usuario> -p <nome_do_banco> <  
database/schema.sql>
```

2.2.3 Definição de Variáveis de Ambiente

Crie um arquivo ".env" ou configure as variáveis diretamente no ambiente do servidor com as seguintes informações:

```
DB_HOST=endereco_do_banco  
DB_USER=usuario_do_banco  
DB_PASS=senha_do_banco  
DB_NAME=nome_do_banco
```

2.2.4 Instalação de Dependências

Prepare o ambiente Python isolado e instale os pacotes requeridos:

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

2.2.5 Execução do Processo ETL (Carga Inicial)

O banco de dados deve ser populado antes da inicialização da aplicação web. Execute os scripts ETL_ consolidados:

```
python etl_deputados.py
python etl_discursos.py
python etl_proposicoes.py
python etl_proposicoestema.py
python etl_temas.py
python etl_presencas.py
python etl_despesas.py
```

Nota: Recomenda-se o agendamento rotineiro deste script (exemplo: via crontab) para a manutenção e atualização contínua dos dados.

2.2.6 Inicialização da Aplicação

Inicie o servidor de aplicação. No terminal da IDE, digite: ".\env\Scripts\python.exe app.py" . A aplicação irá iniciar em um endereço fornecido no terminal. Abra o navegador e acesse o endereço fornecido no terminal(CRTL+CLICK). Para encerrar a aplicação, pressione CRTL+C no terminal onde o servidor está em execução.